CAB SYSTEM

HƯỚNG DẪN THỰC HÀNH POSTMAN VÀ TRẢ LỜI BẢO VỆ — 30 CÂU

Giữ format tài liệu mẫu • Cập nhật nội dung ngày 27/10/2026

QUAN TRỌNG: Các HTTP status và trạng thái trong tài liệu là KẾT QUẢ KỲ VỌNG. Việc biên soạn PDF không phải kiểm thử đủ 30 câu. Customer đăng ký không có OTP hoặc token; Driver cần OTP. Chỉ đăng nhập mới cấp token.

CHUẨN BỊ MÔI TRƯỜNG

1. Mở Git Bash ở root repository. Khi cần khởi động/cập nhật hệ thống, chạy lần lượt; không chạy reset hoặc xóa volume để làm bài:

```
npm test
npm run check
docker compose up -d --build
docker compose ps -a
```

2. Chưa có Admin thì bootstrap một lần. Nếu Admin seed cũ thiếu email hoặc sai mật khẩu, dùng lệnh sửa riêng ở dưới. Chọn đúng một lệnh theo tình huống; không seed Customer/Driver mẫu thay cho phần nhập tay.

```
docker compose exec -T auth_service node scripts/seed.cjs \
  auth_service --admin-only --password quang123

docker compose exec -T auth_service node scripts/seed.cjs \
  auth_service --admin-only --repair-admin --password quang123
```

Admin đăng nhập bằng admin@cab.local / quang123 nếu đã bootstrap/sửa theo lệnh trên. Seed thông thường không đổi mật khẩu Admin đã có. PowerShell dùng npm.cmd nếu npm.ps1 bị chặn; dấu \ cuối dòng là nối lệnh của Git Bash.

3. Postman: New HTTP Request → chọn Method → nhập URL → Body raw/JSON → Send. GET dùng Body none. Không dùng biến hoặc script tự điền trong bài nhập tay.

```
Base URL: http://localhost:3000
Content-Type: application/json
Authorization: Bearer TOKEN_THAT
```

Các đường dẫn /api/v1/... phải nối vào Base URL. Dùng No Auth và tự nhập header Bearer, hoặc chọn Bearer Token rồi dán token — không đặt cả hai. Tài khoản Admin, Customer, Driver dùng token riêng.

Ghi tay: C_TOKEN, C2_TOKEN, D_TOKEN, A_TOKEN, CUSTOMER_ID, DRIVER_ID, BOOKING_ID, INVITATION_ID, TRIP_ID, PAYMENT_ID. Chữ in hoa là chỗ thay bằng giá trị thật; cid/did không thay ObjectId.

THỨ TỰ DEMO HỢP LÝ

5–8 → 9–11 → 21–22 → 23/13 → 15–17 → 19–20 → 18/14 → 12/24–30. Làm callback replay của câu 30 ngay sau câu 19 để không hết hạn timestamp.

CÂU 01 | LÝ THUYẾT

Mô tả kiến trúc tổ chức source code

Thao tác minh họa: `ls gateway services proto infra scripts`; mở cây source

• Mở VS Code tại root repository; chỉ ra gateway/, services/, proto/, infra/ và scripts/.

• Trình bày 8 service: Auth, Customer, Driver, Booking, Trip, Payment, Notification, Rating.

CÂU TRẢ LỜI BẢO VỆ

"Gateway là cổng REST; có đúng 8 service theo bounded context. Mỗi service sở hữu dữ liệu của mình."

CÂU 02 | LÝ THUYẾT

Kiểm tra .gitignore và .env trên github

Thao tác minh họa: `git check-ignore .env`; `git ls-files -- .env .env.example`; `git log --all --oneline -- .env`; mở đúng branch/history GitHub

• Mở .gitignore, .env.example; kiểm tra git và lịch sử Git.

```
git check-ignore .env
git ls-files -- .env .env.example
git log --all --oneline -- .env
```

• Kiểm tra branch đang bảo vệ và lịch sử GitHub; ignore không xóa secret đã commit.

CÂU TRẢ LỜI BẢO VỆ

".env giữ secret local; Git theo dõi template. Ignore không tự xóa file đã tracked hoặc lịch sử; kiểm local chưa thay cho kiểm GitHub."

CÂU 03 | LÝ THUYẾT

Mô tả nhiệm vụ gateway trong hệ thống

Thao tác minh họa: Mở gateway/index.cjs và router

• Mở gateway/index.cjs và router, lần lượt chỉ ra validation, JWT, role, rate limit, mapping REST → gRPC.

• Cho thấy quyền sở hữu dữ liệu vẫn được service kiểm tra.

CÂU TRẢ LỜI BẢO VỆ

"Gateway kiểm schema/JWT/role, áp dụng rate limit Redis, gọi gRPC và chuẩn hóa lỗi. Ownership vẫn được kiểm ở service."

CÂU 04 | LÝ THUYẾT

Mô tả IPC của các microservice

Thao tác minh họa: Mở proto, rpc-security, event-bus và topology

• Mở proto/, cấu hình RPC security, event bus và RabbitMQ topology.

• Giải thích: gRPC yêu cầu-đáp ứng đồng bộ; RabbitMQ chuyển sự kiện bất đồng bộ.

CÂU TRẢ LỜI BẢO VỆ

"Đồng bộ dùng gRPC khi cần kết quả ngay; bất đồng bộ dùng RabbitMQ cho profile/notification/release. Không dùng REST nội bộ hay thêm Kafka."

CÂU 05 | THỰC HÀNH

Compose hệ thống và liệt kê các container

```
docker compose config --quiet
docker compose ps -a
```

Bước 1. Xác nhận Gateway, 8 services, MongoDB, Redis và RabbitMQ đang hoạt động; xem infra_init là job bootstrap exit 0.

CÂU TRẢ LỜI BẢO VỆ

“Compose chạy Gateway, 8 service, MongoDB/Redis/RabbitMQ. infra_init là job bootstrap exit 0, không phải service nghiệp vụ thứ 9.”

CÂU 06 | THỰC HÀNH

POSTMAN: API health check

Bước 1. GET /health → 200 nếu Gateway chạy.

Bước 2. GET /ready → 200 nếu dependencies sẵn sàng; 503 nếu lỗi.

Bước 3. GET /health/services → kiểm tra trạng thái từng service.

CÂU TRẢ LỜI BẢO VỆ

“Health là tiến trình; ready kiểm dependencies. Inventory có 12 thành phần vận hành, không tính bootstrap job; dependency lỗi phải trả 503 ở readiness.”

CÂU 07 | THỰC HÀNH

Kiểm tra hệ thống Kafka / Rabbit MQ

```
docker compose exec -T rabbitmq rabbitmqctl list_exchanges -p cab name type durable
docker compose exec -T rabbitmq rabbitmqctl list_bindings -p cab \
  source_name destination_name routing_key
docker compose exec -T rabbitmq rabbitmqctl list_queues -p cab \
  name messages_ready messages_unacknowledged consumers
docker compose exec -T rabbitmq rabbitmqctl list_consumers -p cab
```

Bước 1. Vhost cab: cab.events (topic), cab.dead (direct); bốn queue chính customer.events, driver.events, booking.events, notification.events; 12 retry queue và 4 DLQ.

Bước 2. Sau câu 9, account.registered tạo Customer profile; event này không gửi notification. Đối chiếu Consumer/outbox/inbox, không chỉ số message còn lại.

Bước 3. Event-bus await topology trước consume/publish và khi reconnect. Queue trống sau ACK là bình thường; infra_init exit 0 không bảo đảm topology còn nguyên.

CÂU TRẢ LỜI BẢO VỆ

“DB/outbox cùng transaction; publish confirm; inbox chống trùng, ACK sau commit. Retry 1/5/15 giây rồi DLQ. Recovery, replay và DLQ cần bằng chứng chạy riêng.”

CÂU 08 | THỰC HÀNH

Kiểm tra mọi request đều phải đi qua Gateway

Bước 1. Đảm bảo tất cả HTTP Request của Client dùng Gateway http://localhost:3000.

```
docker compose port auth_service 50051
```

Bước 2. Mở Docker Compose và gRPC security middleware; cổng gRPC không được publish ra host, metadata nội bộ được xác minh.

CÂU TRẢ LỜI BẢO VỆ

“gRPC không publish ra host. Gateway tự đặt internal metadata, service kiểm secret/peer allowlist; không tin role/ID tự khai từ client.”

CÂU 09 | THỰC HÀNH

POSTMAN: Đăng ký tài khoản khách hàng

Bước 1. POST /api/v1/auth/register — No Auth, Body raw JSON:

```
{
  "email": "cus01@gmail.com",
  "password": "quang123",
  "phone": "0909212109"
}
```

Bước 2. Kỳ vọng 201, chỉ gồm accountId, role=CUSTOMER, cid do server cấp. Không có token, profileStatus hoặc bước OTP. Không nhập role/name/cid.

Bước 3. Đăng nhập riêng theo câu 10 để lấy C_TOKEN, rồi GET /api/v1/customers/me. Ghi CUSTOMER_ID từ customerId; không dùng cid thay cho ObjectId trong URL.

Bước 4. Nếu 503 PROFILE_NOT_READY, chờ Consumer rồi Send lại GET; không đăng ký lại. Nếu 409, email/phone đã tồn tại: đăng nhập tài khoản đó hoặc chọn dữ liệu mới.

CÂU TRẢ LỜI BẢO VỆ

“Register chỉ tạo tài khoản và trả định danh, không tự đăng nhập. Customer không cần OTP. Profile tạo bất đồng bộ qua account.registered; thiếu profile tạm thời không có nghĩa đăng ký thất bại.”

CÂU 10 | THỰC HÀNH

POSTMAN: Đăng nhập khách hàng

Bước 1. POST /api/v1/auth/login — No Auth:

```
{
  "email": "cus01@gmail.com",
  "password": "quang123"
}
```

Bước 2. Kỳ vọng 200 với accessToken và account tối giản. Ví dụ cấu trúc phản hồi; các dấu ... là chỗ thay bằng giá trị thật:

```
{
  "accessToken": "TOKEN_THAT",
  "account": {"accountId": "OBJECT_ID_THAT", "role": "CUSTOMER", "cid": "c..."}
}
```

Bước 3. Lưu accessToken là C_TOKEN. JSON đăng nhập không trả status: ACTIVE, expiresAt hoặc tokenType. JWT vẫn có exp/iat nội bộ; không xóa thời hạn token.

Bước 4. Giữ email, đổi password thành wrong123 rồi Send lại → kỳ vọng 401. Khôi phục mật khẩu đúng trước các câu tiếp theo.

CÂU TRẢ LỜI BẢO VỆ

“Chỉ Login cấp token sau khi kiểm Argon2id và trạng thái tài khoản nội bộ. Response được rút gọn; token vẫn phải được xác minh chữ ký, issuer, audience và thời hạn.”

CÂU 11 | THỰC HÀNH

POSTMAN: Lấy thông tin khách hàng với mã số

Bước 1. Đăng ký Customer thứ hai (email, phone khác), login lấy C2_TOKEN.

Bước 2. GET /api/v1/customers/CUSTOMER_ID — C_TOKEN → 200.

Bước 3. Giữ URL, thay C2_TOKEN → kỳ vọng 403.

Bước 4. ADMIN có thể GET /api/v1/admin/customers?page=1&limit=10 bằng A_TOKEN để xem danh sách, rồi đổi page=2. Hồ sơ cũ có thể thiếu cid. Không có DELETE /admin/customers hoặc query customerId để xóa.

CÂU TRẢ LỜI BẢO VỆ

“Token hợp lệ chưa đủ: service kiểm đúng chủ hồ sơ; khác chủ bị 403.”

CÂU 12 | THỰC HÀNH

POSTMAN: Lấy thông tin tài xế với mã số

Bước 1. Hoàn thành câu 21 để có DRIVER_ID và D_TOKEN.

Bước 2. GET /api/v1/drivers/me — D_TOKEN.

Bước 3. GET /api/v1/drivers/DRIVER_ID — D_TOKEN hoặc A_TOKEN; không trả license cho Customer ngoài quyền.

Bước 4. So sánh public driver snapshot trong Booking sau khi đã phân công.

CÂU TRẢ LỜI BẢO VỆ

“Hồ sơ riêng và license chỉ owner/ADMIN. Nearby/Trip chỉ trả public snapshot, không lộ giấy phép.”

CÂU 13 | THỰC HÀNH

POSTMAN: Liệt kê danh sách tài xế tại khu vực

Bước 1. Chuẩn bị ≥5 Driver có trạng thái/điều kiện khác nhau; chỉ các Driver hợp lệ mới ở danh sách nearby.

Bước 2. PUT /api/v1/drivers/me/location — D_TOKEN.

```
{
  "locationCode": "A"
}
```

Bước 3. PUT /api/v1/drivers/me/availability — D_TOKEN.

```
{
  "isOnline": true
}
```

Bước 4. GET /api/v1/drivers/nearby?locationCode=A&page=1&limit=2 — C_TOKEN; thử page=2.

Bước 5. Vị trí Driver phải mới ≤120 giây, cách ≤1 km; APPROVED/ACTIVE/online/có xe/không bận.

CÂU TRẢ LỜI BẢO VỆ

“A là điểm mẫu trong DB. Chỉ xét <=1 km, vị trí mới <=120 giây, APPROVED/ACTIVE/online/có xe/không bận. Không phải cả 5 fixture đều được trả.”

CÂU 14 | THỰC HÀNH

POSTMAN: Liệt kê danh sách các booking của Customer

Bước 1. Chuẩn bị ≥5 Booking của cùng Customer; có thể tạo các key b11...b15 khi không có Driver nhận.

Bước 2. GET /api/v1/bookings?page=1&limit=2 — C_TOKEN.

Bước 3. GET /api/v1/bookings?page=2&limit=2 — C_TOKEN.

Bước 4. Kiểm totalItems, ownership, thứ tự giảm dần; trang vượt phạm vi trả items rỗng.

CÂU TRẢ LỜI BẢO VỆ

“History lọc ownership và phân trang, sort createdAt rồi bookingId giảm dần; trang rỗng trả items rỗng.”

CÂU 15 | THỰC HÀNH

POSTMAN: Đặt xe

Bước 1. Bảo đảm Driver phù hợp đang online và vị trí A còn mới.

Bước 2. POST /api/v1/bookings — C_TOKEN; thêm header Idempotency-Key: b01.

```
{
  "pickupCode": "A",
  "destinationCode": "B"
}
```

Bước 3. Kỳ vọng 201/SEARCHING/BIKE, ghi BOOKING_ID. Không nhập amount/customerId/vehicleType.

CÂU TRẢ LỜI BẢO VỆ

"Server đọc tọa độ DB, tự gán BIKE và chụp snapshot khoảng cách/giá. Không nhận amount/customerId/vehicleType tự chọn."

CÂU 16 | THỰC HÀNH

POSTMAN: Tài xế nhận chuyến

Bước 1. Ngay sau câu 15: GET /api/v1/drivers/me/invitations?status=PENDING&page=1&limit=50 — D_TOKEN.

Bước 2. Tìm đúng BOOKING_ID và ghi INVITATION_ID; lời mời chỉ tồn tại 30 giây.

Bước 3. POST /api/v1/invitations/INVITATION_ID/response — D_TOKEN.

```
{
  "decision": "ACCEPT"
}
```

Bước 4. Kỳ vọng ACCEPTED/ASSIGNED, ghi TRIP_ID.

CÂU TRẢ LỜI BẢO VỆ

"Mỗi Booking mời tuần tự, 30 giây/offer, tối đa 5 ứng viên ban đầu. Reserve Driver và tạo/activate Trip trước khi hoàn tất phân công; DB unique chống nhận hai chuyến."

CÂU 17 | THỰC HÀNH

POSTMAN: Cập nhật trạng thái chuyến

Bước 1. POST /api/v1/trips/TRIP_ID/arrival — D_TOKEN.

```
{}
```

Bước 2. Kỳ vọng ARRIVED.

Bước 3. POST /api/v1/trips/TRIP_ID/start — D_TOKEN, body {} → IN_PROGRESS.

Bước 4. PUT /api/v1/trips/TRIP_ID/location — D_TOKEN.

```
{
  "locationCode": "B"
}
```

Bước 5. POST /api/v1/trips/TRIP_ID/completion — D_TOKEN, body {} → COMPLETED và fareAmount.

CÂU TRẢ LỜI BẢO VỆ

"Không bỏ qua hoặc lùi trạng thái. Driver phải là người được phân công. Cước dựa snapshot Booking, không lấy số tiền khách nhập."

CÂU 18 | THỰC HÀNH

POSTMAN: Hủy chuyến

Bước 1. Tạo Booking riêng b02, cho Driver nhận để Trip ở ASSIGNED/ARRIVED. Chưa bắt đầu hoặc hoàn thành Trip; đây là fixture riêng, không dùng Trip đã thanh toán ở câu 19.

Bước 2. POST /api/v1/bookings/BOOKING_ID/cancellation-preview — C_TOKEN.

```
{
  "reason": "Doi ke hoach"
}
```

Bước 3. Kỳ vọng allowed=true, feeAmount=0 VND.

Bước 4. POST /api/v1/bookings/BOOKING_ID/cancellation — C_TOKEN.

```
{
  "reason": "Doi ke hoach",
  "confirmed": true
}
```

Bước 5. Kỳ vọng CANCELED, kiểm Trip/Notification và Driver release.

CÂU TRẢ LỜI BẢO VỆ

"Customer chỉ hủy SEARCHING hoặc Trip ASSIGNED/ARRIVED. Server kiểm lại lúc xác nhận, ghi người/lý do/thời gian, đóng offer hoặc release; không tạo Payment hủy."

CÂU 19 | THỰC HÀNH

POSTMAN: Thanh toán online

Bước 1. Chỉ tạo Payment khi TRIP_ID đã COMPLETED. C_TOKEN gọi POST /api/v1/trips/TRIP_ID/payments, header Idempotency-Key: p01, body {}.

Bước 2. Kỳ vọng 201/PENDING/MOCK_ONLINE. Ghi PAYMENT_ID, referenceCode và amount đúng từ response. Không tự tính hoặc nhập số tiền khác.

Bước 3. Tại Git Bash, lấy Unix timestamp hiện tại; không dùng một timestamp cũ trong ví dụ:

```
date +%s
```

Bước 4. Soạn JSON callback một dòng từ dữ liệu thật. Ví dụ dưới chỉ minh họa cấu trúc; thay PAYMENT_ID, REFERENCE_CODE, amount, timestamp trước khi ký. eventId phải mới cho callback gốc.

```
{
  "eventId": "cb01",
  "paymentId": "PAYMENT_ID",
  "referenceCode": "REFERENCE_CODE",
  "amount": 15000,
  "currency": "VND",
  "result": "COMPLETED",
  "timestamp": 1790000000
}
```

```
node --env-file=.env scripts/sign-manual-callback.cjs \
  TIMESTAMP 'RAW_JSON_MOT_DONG_DA_THAY_DU_GIA_TRI'
```

Bước 5. POST /api/v1/payments/mock/callback — No Auth. Tự nhập Content-Type: application/json, X-Mock-Timestamp và X-Mock-Signature từ kết quả ký. Dán đúng từng byte chuỗi đã ký vào raw body; không Beautify hoặc thêm newline. Send trong 5 phút.

Bước 6. C_TOKEN GET /api/v1/payments/PAYMENT_ID và /api/v1/trips/TRIP_ID/payment-summary → kỳ vọng COMPLETED/hasCompleted=true. Chuẩn bị replay câu 30 ngay trong hạn 5 phút; giữ nguyên callback gốc.

CÂU TRẢ LỜI BẢO VỆ

"Tạo Payment PENDING chưa phải thanh toán thành công. Callback phải có chữ ký, timestamp và dữ liệu khớp. Trip và Payment có trạng thái riêng; Driver không phải chờ Payment mới được giải phóng."

CÂU 20 | THỰC HÀNH

POSTMAN: Đánh giá chuyến đi

Bước 1. POST /api/v1/trips/TRIP_ID/reviews — C_TOKEN, Trip COMPLETED.

```
{
  "rating": 5,
  "comment": "Tot"
}
```

Bước 2. Kỳ vọng 201. GET cùng URL xem Review.

Bước 3. Gửi lại review cùng Trip → 409. Thử rating=6 ở Trip hợp lệ khác chưa review → 400.

CÂU TRẢ LỜI BẢO VỆ

“Chỉ Customer của Trip COMPLETED được đánh giá, một review mỗi Trip. Không cần Payment thành công mới cho review.”

CÂU 21 | THỰC HÀNH

POSTMAN: Đăng ký tài xế

Bước 1. POST /api/v1/auth/register — No Auth, lần 1.

```
{
  "email": "dri01@gmail.com",
  "password": "quang123",
  "phone": "0909212110",
  "plateNumber": "59A12345"
}
```

Bước 2. Kỳ vọng 202/OTP_REQUIRED, ghi mockCode thực tế (OTP giả lập local).

Bước 3. POST cùng URL lần 2, giữ 4 trường và thêm code từ mockCode.

```
{
  "email": "dri01@gmail.com",
  "password": "quang123",
  "phone": "0909212110",
  "plateNumber": "59A12345",
  "code": "MA_OTP_THAT"
}
```

Bước 4. Kỳ vọng 201, Account/did được tạo; login /api/v1/auth/login để lấy D_TOKEN.

Bước 5. PATCH /api/v1/drivers/me — D_TOKEN.

```
{
  "licenseNumber": "DEMO01"
}
```

Bước 6. POST /api/v1/drivers/me/submissions — D_TOKEN, body {} → PENDING.

Bước 7. OTP_REQUIRED là đang chờ xác thực OTP; PENDING sau Submit là hồ sơ chờ Admin duyệt. Hai trạng thái khác nhau. Register hoàn tất không trả token; Driver đăng nhập riêng như Customer.

CÂU TRẢ LỜI BẢO VỆ

“Lần đầu chỉ tạo challenge, chưa có Account. Lần hai kiểm OTP gắn dữ liệu đăng ký rồi mới tạo Account/did. Sau đó gửi hồ sơ PENDING mới đủ tiêu chí; OTP không phải GPLX hay biển số.”

CÂU 22 | THỰC HÀNH

POSTMAN: Duyệt hồ sơ tài xế

Bước 1. POST /api/v1/auth/login — No Auth, email admin@cab.local và password bootstrap, ghi A_TOKEN.

Bước 2. GET /api/v1/admin/drivers?approvalStatus=PENDING&page=1&limit=10 — A_TOKEN.

Bước 3. GET /api/v1/drivers/DRIVER_ID — A_TOKEN.

Bước 4. POST /api/v1/admin/drivers/DRIVER_ID/approval-decisions — A_TOKEN.

```
{
  "decision": "APPROVED"
}
```

Bước 5. Kỳ vọng APPROVED; Driver GET /api/v1/notifications?page=1&limit=10.

Bước 6. REJECTED chỉ minh họa bằng hồ sơ PENDING khác, có reason.

CÂU TRẢ LỜI BẢO VỆ

“Chỉ ADMIN quyết định trên hồ sơ PENDING; Driver không tự gửi approvalStatus qua PATCH. Event thông báo kết quả, không tự online.”

CÂU 23 | THỰC HÀNH

POSTMAN: Bật/tắt trạng thái nhận chuyến

Bước 1. Driver cần đã được duyệt trước khi online.

Bước 2. PUT /api/v1/drivers/me/location — D_TOKEN.

```
{
  "locationCode": "A"
}
```

Bước 3. PUT /api/v1/drivers/me/availability — D_TOKEN.

```
{
  "isOnline": true
}
```

Bước 4. PUT cùng URL với isOnline=false để offline.

```
{
  "isOnline": false
}
```

Bước 5. GET /api/v1/drivers/me — D_TOKEN để xác nhận trạng thái.

CÂU TRẢ LỜI BẢO VỆ

“Online cần account ACTIVE, hồ sơ APPROVED, xe và vị trí mới; offline ngừng nhận mới, không tự hủy Trip đang chạy.”

CÂU 24 | THỰC HÀNH

POSTMAN: Data encryption at rest

Bước 1. Kết nối MongoDB bằng đúng owner Auth; nhập mật khẩu DB local tại prompt, không dùng mật khẩu tài khoản Postman cho MongoDB:

```
docker compose exec mongo mongosh --host 127.0.0.1 \
  --username cab_auth --authenticationDatabase admin --password

use auth_db
db.accounts.findOne({email:"cus01@gmail.com"},{_id:0,email:1,passwordHash:1})
exit
```

Bước 2. Kết nối owner Driver, xem bản mã giấy phép của Driver thật đã tạo:

```
docker compose exec mongo mongosh --host 127.0.0.1 \
  --username cab_driver --authenticationDatabase admin --password

use driver_db
db.driver_profiles.findOne({did:"DID_THAT"},{_id:0,did:1,licenseNumberEncrypted:1})
exit
```

Bước 3. Đối chiếu scripts/driver-crypto.cjs và password hash trong source. Username/database theo cấu hình local nếu đã đổi. Chỉ xem dữ liệu demo; không đưa .env, khóa hoặc bản dump tài khoản lên màn hình/GitHub.

CÂU TRẢ LỜI BẢO VỆ

"Password được hash Argon2id có salt, không giải mã. Giấy phép demo dùng AES-256-GCM; khóa nằm ngoài database. Đây là bảo vệ trường dữ liệu, không tự chứng minh toàn bộ ổ đĩa MongoDB đã mã hóa; cần đối chiếu đúng phạm vi mục chấm."

CÂU 25 | THỰC HÀNH

POSTMAN: SQL injection attempt

Bước 1. POST /api/v1/auth/login — No Auth. Thử chuỗi SQL-like:

```
{"email":"' OR 1=1 --","password":"quang123"}
```

Bước 2. Kỳ vọng 400 do email sai định dạng; không có accessToken, không bypass đăng nhập hoặc lộ dữ liệu DB.

Bước 3. Giữ URL và Method, thử NoSQL object injection:

```
{"email":{"$ne":null},"password":"quang123"}
```

Bước 4. Kỳ vọng 400 vì email không phải string. Dùng thêm một tài khoản đúng để đối chiếu Login 200; không sửa validation cho payload tấn công đi qua.

CÂU TRẢ LỜI BẢO VỆ

"Dùng schema, whitelist field, scalar validation và query có cấu trúc. Không ghép chuỗi query. MongoDB vẫn cần chống NoSQL injection; không trả token cho đầu vào tấn công."

CÂU 26 | THỰC HÀNH

POSTMAN: XSS input test

Bước 1. PATCH /api/v1/customers/me — C_TOKEN, raw JSON:

```
{"name":"<script>alert(1)</script>"}
```

Bước 2. Kỳ vọng 400; GET lại để xác nhận tên không bị thay bằng script. Postman không chạy JavaScript như trình duyệt, nên cần browser harness riêng nếu phải chứng minh script không execute.

CÂU TRẢ LỜI BẢO VỆ

"Backend từ chối markup, trả JSON. Giao diện phải escape đúng ngữ cảnh hoặc dùng textContent. HTTP 400 trong Postman chưa đủ chứng minh toàn bộ kiểm thử XSS trên trình duyệt."

CÂU 27 | THỰC HÀNH

POSTMAN: JWT tampering

Bước 1. GET /api/v1/customers/me với C_TOKEN đúng → 200.

Bước 2. Sao chép JWT, sửa ký tự chữ ký hoặc payload rồi dùng Bearer token đã sửa.

Bước 3. GET lại → kỳ vọng 401. Decode JWT không thay cho verify chữ ký.

CÂU TRẢ LỜI BẢO VỆ

"Decode payload không thay verify. Service kiểm HS256/signature/issuer/audience/expiry/sub; đổi role thành ADMIN không có quyền nếu chữ ký sai."

CÂU 28 | THỰC HÀNH

POSTMAN: Unauthorized API access

Bước 1. PATCH /api/v1/drivers/me — C_TOKEN.

```
{
  "licenseNumber": "DEMO02"
}
```

Bước 2. Kỳ vọng 403 vì sai role.

Bước 3. GET /api/v1/bookings/BOOKING_ID — C2_TOKEN, Booking của C1 → 403.

Bước 4. GET /api/v1/payments/PAYMENT_ID — C2_TOKEN, Payment của C1 → 403.

Bước 5. Bỏ Bearer token rồi GET dữ liệu bảo vệ → 401.

CÂU TRẢ LỜI BẢO VỆ

"401 là thiếu/sai token; 403 là token hợp lệ nhưng sai role/ownership. Không lấy ID trong body để cấp quyền."

CÂU 29 | THỰC HÀNH

POSTMAN: Rate limit attack

Bước 1. Chờ ≥60 giây không tạo Booking bằng Customer trước khi thử.

Bước 2. POST /api/v1/bookings — C_TOKEN, Idempotency-Key: rate01.

```
{
  "pickupCode": "A",
  "destinationCode": "B"
}
```

Bước 3. Bấm Send 11 lần trong 60 giây, giữ nguyên body/key; lần 11 kỳ vọng 429, xem Retry-After.

Bước 4. Nếu cần chứng minh >1000 req/s, dùng script đo tải riêng và ghi số liệu THỰC, không suy từ 11 lần Send.

Bước 5. Các ngưỡng 10 request/60 giây là mặc định local. Kiểm tra cấu hình rate limit trước buổi demo; nếu đã đổi ngưỡng thì dùng kết quả kỳ vọng tương ứng. Không xóa Redis counter để làm bài đạt.

CÂU TRẢ LỜI BẢO VỆ

"Redis giữ counter/TTL; cả replay cũng tính request. 11 lần Send chứng minh rate limit, còn >1000 req/s phải có số đo thật từ tool tải; chưa đo thì không tự nói PASS phần tải."

CÂU 30 | THỰC HÀNH

POSTMAN: Replay attack (idempotency)

Bước 1. POST /api/v1/bookings — C_TOKEN, Idempotency-Key: b01.

```
{
  "pickupCode": "A",
  "destinationCode": "B"
}
```

Bước 2. Replay cùng key/body → kỳ vọng 200, cùng bookingId/snapshot.

Bước 3. Giữ key b01, đổi body khác → kỳ vọng 409.

```
{
  "pickupCode": "B",
  "destinationCode": "A"
}
```

Bước 4. Replay POST /api/v1/trips/TRIP_ID/payments cùng key p01, body {} → cùng Payment ID.

Bước 5. Replay callback với NGUYÊN eventId/body/timestamp/signature từ câu 19 → duplicate=true; không ký lại.

Bước 6. ADMIN GET /api/v1/admin/payments?tripId=TRIP_ID&page=1&limit=10 để đối chiếu attempts.

Bước 7. Callback phải còn trong hạn timestamp. Nếu đã quá 5 phút thì bị từ chối là đúng; dùng một Payment fixture mới để chuẩn bị lại bài, không sửa thời gian DB hoặc tắt kiểm tra chữ ký.

CÂU TRẢ LỜI BẢO VỆ

“Cùng key/payload trả cùng ID và snapshot, khác payload409. Callback xác minh signature trước kiểm duplicate; event cũ đúng trả duplicate=true, kết quả trái ngược bị từ chối; không double charge.”

LƯU Ý CUỐI CÙNG

Dùng email/phone/biển số mới khi dữ liệu demo đã tồn tại. 409 không có nghĩa phải xóa database. Không có API DELETE /admin/customers; thao tác bảo trì dữ liệu là quy trình riêng, không thực hiện theo tài liệu này.

Lịch sử Booking của Customer cũ không được tính thành lịch sử của Customer mới. Muốn minh họa phân trang câu 14 phải tạo đủ Booking bằng đúng C_TOKEN đang demo.

Các bài recovery/DLQ, replay đồng thời, XSS trên trình duyệt và đo tải >1000 request/giây cần bằng chứng riêng. Chỉ ghi PASS cho phần đã chạy và có output; các phần chưa chạy phải ghi NOT_RUN.

```
docker compose logs --tail=80 gateway auth_service customer_service \
  driver_service booking_service trip_service payment_service
```

Nguồn đối chiếu: PDF mẫu; PHIEU_CHAM_PROJECT.pdf; docs/MANUAL_POSTMAN_30.md; docs/api/01–10 và contracts.md; proto; source hiện hành. Giữ nguyên 8 service, gRPC, RabbitMQ, BIKE và Payment mock sau Trip COMPLETED.